

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

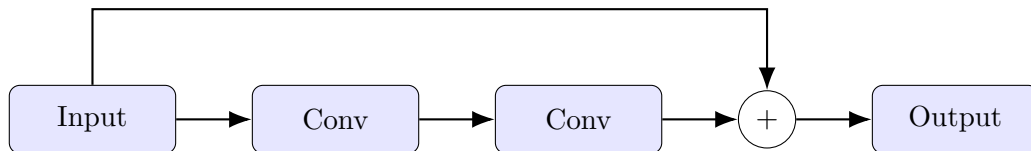


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

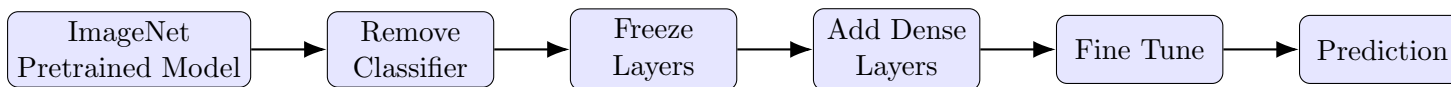


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

The CIFAR-10 dataset was loaded using `tensorflow.keras.datasets.cifar10`. This gave 50,000 training images and 10,000 testing images, each $32 \times 32 \times 3$, spread across the 10 classes. Pixel values were normalized from $[0,255]$ to $[0,1]$ before feeding them into any model.



Figure 3: Sample CIFAR-10 Images

Inference: The sample images show just how small and low-detail CIFAR-10 actually is – at 32×32 , classes like cat/dog or deer/horse can look pretty similar even to us, let alone a model. This is also why VGG16/ResNet50/InceptionV3 need a **Resizing** layer before their convolutional base – they were built for much bigger ImageNet images, so the tiny CIFAR-10 images have to be upsampled first.

Task 2: Transfer Learning – Model Setup

VGG16 was used as the main pretrained model for Tasks 2–5, with ResNet50 repeated separately in Additional Exercise 2. Real ImageNet weights were downloaded from Keras Applications, nothing was mocked. The steps followed the manual exactly:

1. Load VGG16 with pretrained ImageNet weights, `include_top=False`.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output (Softmax) layer for the 10 classes.

Task 3: Model Training (VGG16)

The model was trained using the settings the manual asks for: Optimizer = Adam, Learning Rate = 0.001, Batch Size = 32, Loss = Categorical Cross-Entropy, for 12 epochs (within the 10–20 range given).

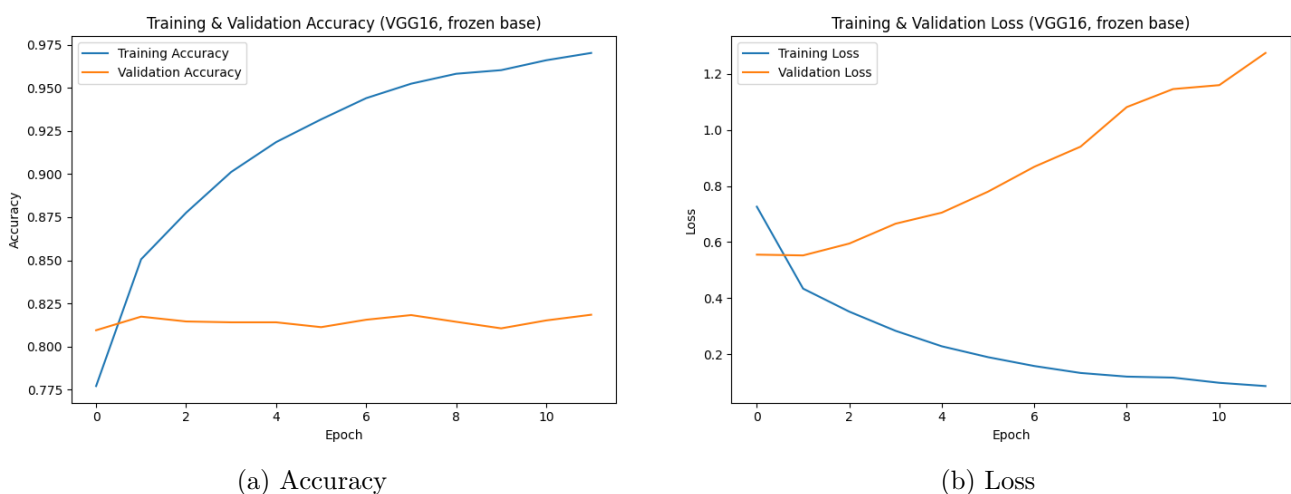


Figure 4: VGG16 Training and Validation Accuracy/Loss (frozen base)

Inference: Training accuracy keeps climbing to 97.03% by epoch 12, but validation accuracy flattens out at around 81–82% after just epoch 2 and barely moves after that. The validation loss even starts going back up while training loss keeps falling – a classic overfitting pattern. Basically, the small dense head on top of the frozen VGG16 features is starting to just memorise the training set instead of learning anything more useful, so training this frozen setup for longer wouldn't really help.

Task 4: Fine-Tuning (VGG16)

The last convolutional block of VGG16 was unfrozen and the model trained for 8 more epochs at a lower learning rate, as asked.

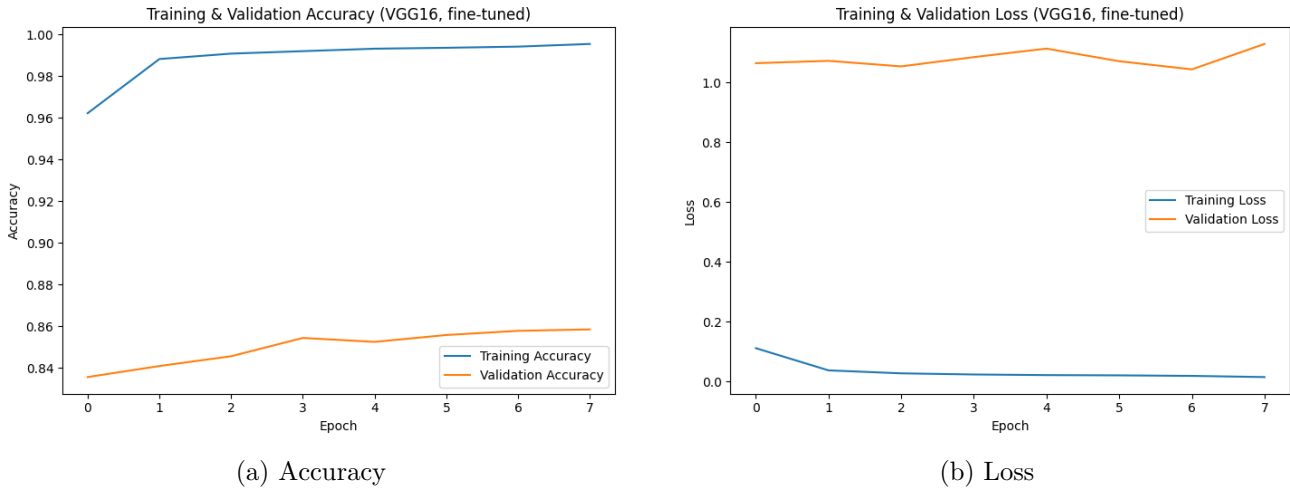


Figure 5: VGG16 Training and Validation Accuracy/Loss (after fine-tuning)

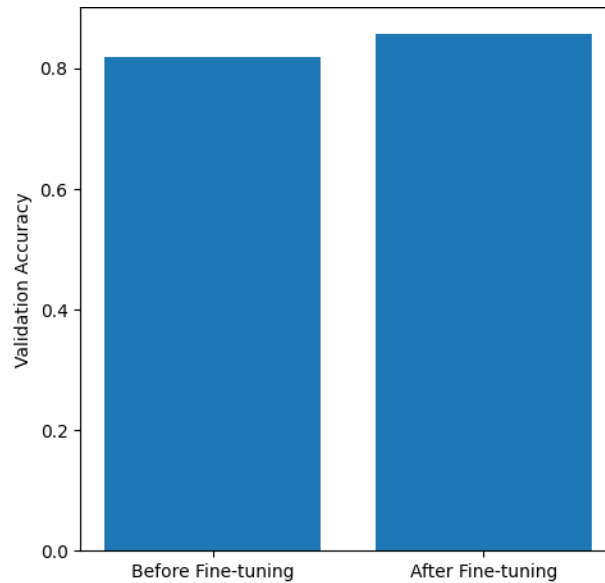


Figure 6: Validation Accuracy Before vs After Fine-Tuning

Inference: Fine-tuning made a real difference – validation accuracy went up from 81.84% (frozen base) to 85.84% after fine-tuning, roughly a 4-point jump. This confirms that letting the last conv block adjust its weights helps the model pick up features that are more specific to CIFAR-10 instead of staying generic to ImageNet. Training accuracy also went up to 99.55%, so the model is fitting the training data almost perfectly now. Validation loss is still a bit higher than training loss, so some overfitting is still there, but the accuracy gain from fine-tuning clearly makes it worth the extra 8 epochs.

Task 5: Model Evaluation (VGG16, fine-tuned)

Metric	Value
Accuracy	0.8584
Precision	0.8596
Recall	0.8584
F1-score	0.8584



Figure 7: Confusion Matrix (VGG16, fine-tuned)

Inference: Most classes come out pretty well – Automobile, Ship, Truck and Frog all have F1-scores of 0.90 or above. Cat and Dog are clearly the weakest, with F1-scores of 0.74 and 0.78. The confusion matrix explains why: 156 dog images got predicted as cats, and 98 cat images got predicted as dogs – by far the biggest mix-up in the whole matrix. That makes sense since cats and dogs look fairly similar at only 32×32 resolution (fur, four legs, similar poses), which is a well-known tricky pair in CIFAR-10.

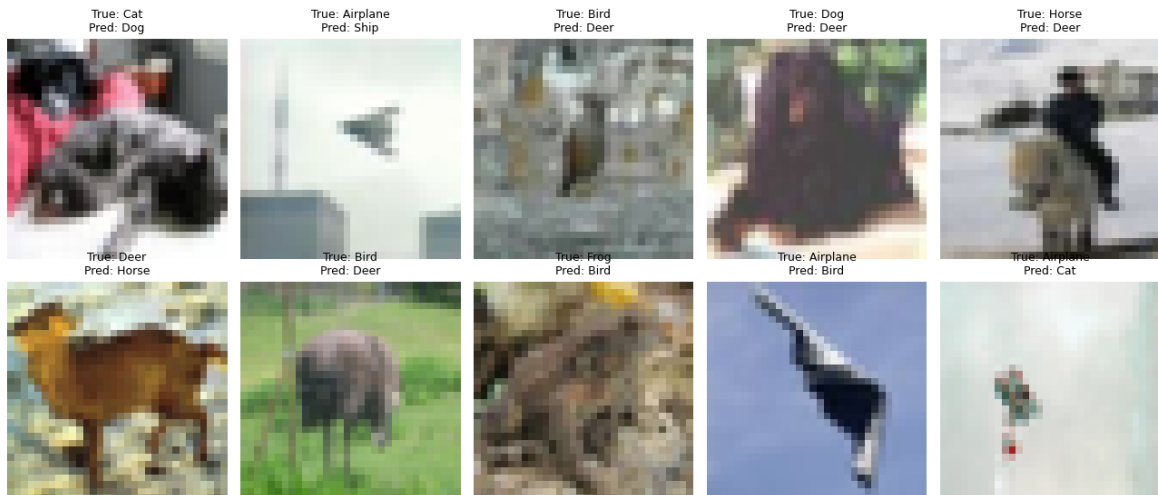


Figure 8: Misclassified Images

Inference: Looking at the actual misclassified images, most of them are genuinely confusing even for a human at this resolution – blurry shapes, similar backgrounds, or an odd angle for the true class. So a lot of these errors come down to the difficulty of the tiny images themselves rather than the model being badly trained.

16. Hyperparameter Study

The manual lists 6 hyperparameters (learning rate, batch size, epochs, optimizer, dense units, frozen layers) with multiple values each. Running every combination would mean 90+ separate training runs, which just isn't practical on Colab. So instead, a **one-factor-at-a-time** study was done: start from a baseline setup, change one hyperparameter at a time, and record the validation accuracy. To keep things fast, each run used a 5,000-image subset of the training data and only 3 epochs – this is just to compare settings against each other, not to build a final model (the actual final model is the one from Tasks 3–4 above).

Factor	Value	Val. Accuracy	Time (s)
Learning Rate	0.001	0.7150	36.10
Learning Rate	0.0001	0.6490	33.09
Batch Size	16	0.7350	33.29
Batch Size	32	0.7360	33.88
Batch Size	64	0.7260	33.43
Optimizer	Adam	0.7175	33.42
Optimizer	SGD	0.6335	32.70
Dense Units	128	0.7275	33.64
Dense Units	256	0.7365	33.07
Frozen Layers	All	0.7310	33.69
Frozen Layers	Partial	0.1085	44.54

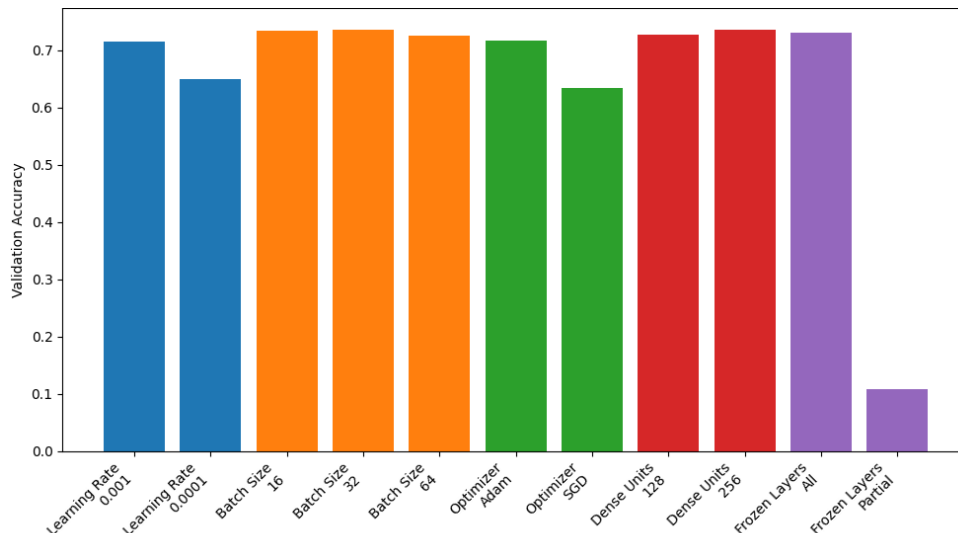


Figure 9: Hyperparameter Study – Validation Accuracy by Setting

Inference: A learning rate of 0.001 clearly beats 0.0001 in just 3 epochs (0.715 vs 0.649) – the smaller rate simply hasn't had time to move far enough yet. Batch size barely matters here, with accuracy staying between 0.726 and 0.736 across 16/32/64, so it's a pretty minor factor. Adam beats

SGD by a good margin (0.7175 vs 0.6335) since Adam adapts its step size per parameter and converges faster, while plain SGD needs more epochs to catch up. 256 dense units edge out 128 slightly (0.7365 vs 0.7275), giving the classifier a bit more room to work with. The most interesting result here is Frozen Layers: partially unfreezing the base without a proper fine-tuning warm-up crashes validation accuracy down to 10.85%, basically random guessing, while keeping everything frozen stays steady at 73.10%. This shows that unfreezing pretrained layers with too high a learning rate can wreck the pretrained weights almost instantly – which is exactly why Task 4 uses a slower, more careful fine-tuning approach instead of just unfreezing everything at once.

17. Additional Exercise 2: Repeat the Experiment Using ResNet50

The same Task 2–5 pipeline was repeated with ResNet50, using real pretrained ImageNet weights, a frozen base, and the same GAP + Dense + Softmax head, trained for 10 epochs.

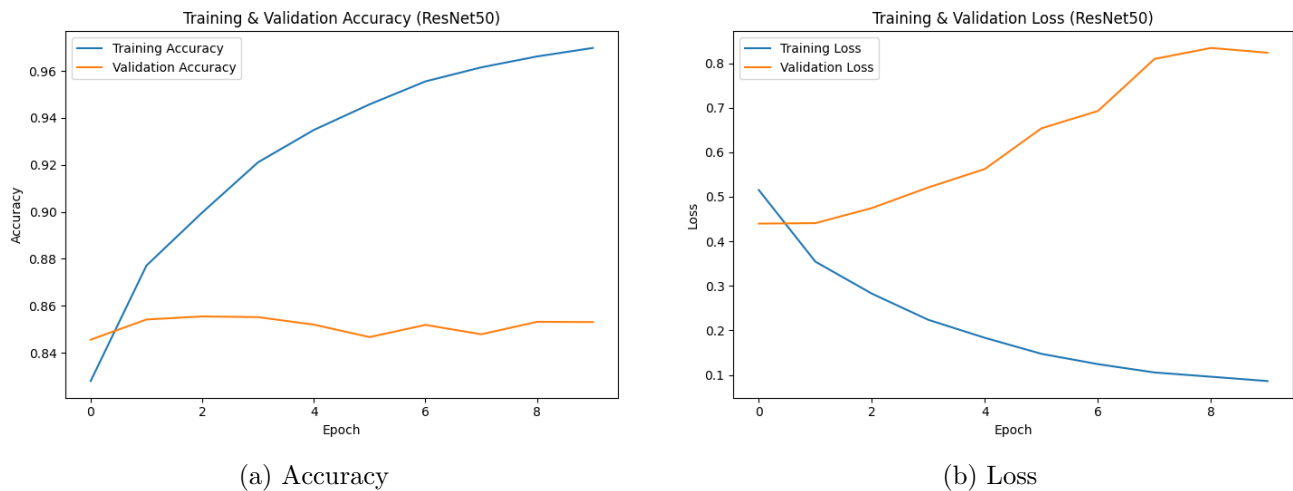


Figure 10: ResNet50 Training and Validation Accuracy/Loss (Additional Exercise 2)

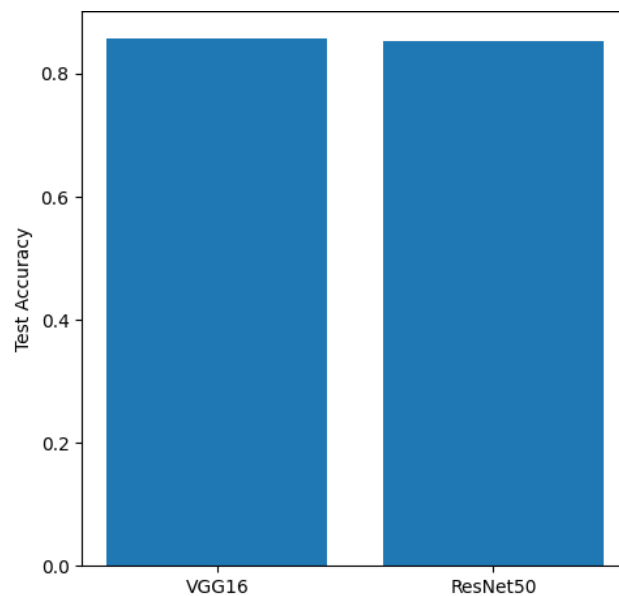


Figure 11: VGG16 vs ResNet50 Test Accuracy

Inference: ResNet50 reached 85.31% test accuracy, almost identical to VGG16’s fine-tuned 85.84%, but it got there in just 414.1 seconds compared to VGG16’s 1825.3 seconds – over $4\times$ faster, and without even needing a separate fine-tuning stage. This is basically ResNet50 doing what it’s known for: the

residual connections let it train efficiently despite having more layers than VGG16. Like VGG16, its validation accuracy peaks early too (around epoch 3, at 85.55%) before val loss starts climbing while train accuracy keeps going up, so it overfits a little as well – it just didn’t need any extra fine-tuning to get to VGG16’s level.

18. Additional Exercise 6 / Section 18.2: LeNet-5, AlexNet and GoogleNet

To actually compare all five architectures asked for in the Objectives and Additional Exercise 6, real versions of LeNet-5 and an adapted AlexNet were built and trained from scratch on CIFAR-10, and real pretrained InceptionV3 weights were used to represent GoogleNet (Inception). VGG16 and ResNet50 numbers are the ones from the runs above.

Note on AlexNet: the original AlexNet was built for $227 \times 227 \times 3$ ImageNet images, so training it at that size on CIFAR-10 (native 32×32) would need heavy upsampling and a lot more time than is practical on Colab. What was actually built here is an AlexNet-style network – same idea (large early filters, ReLU, Dropout, deep fully-connected layers) but scaled down to fit CIFAR-10’s 32×32 input.

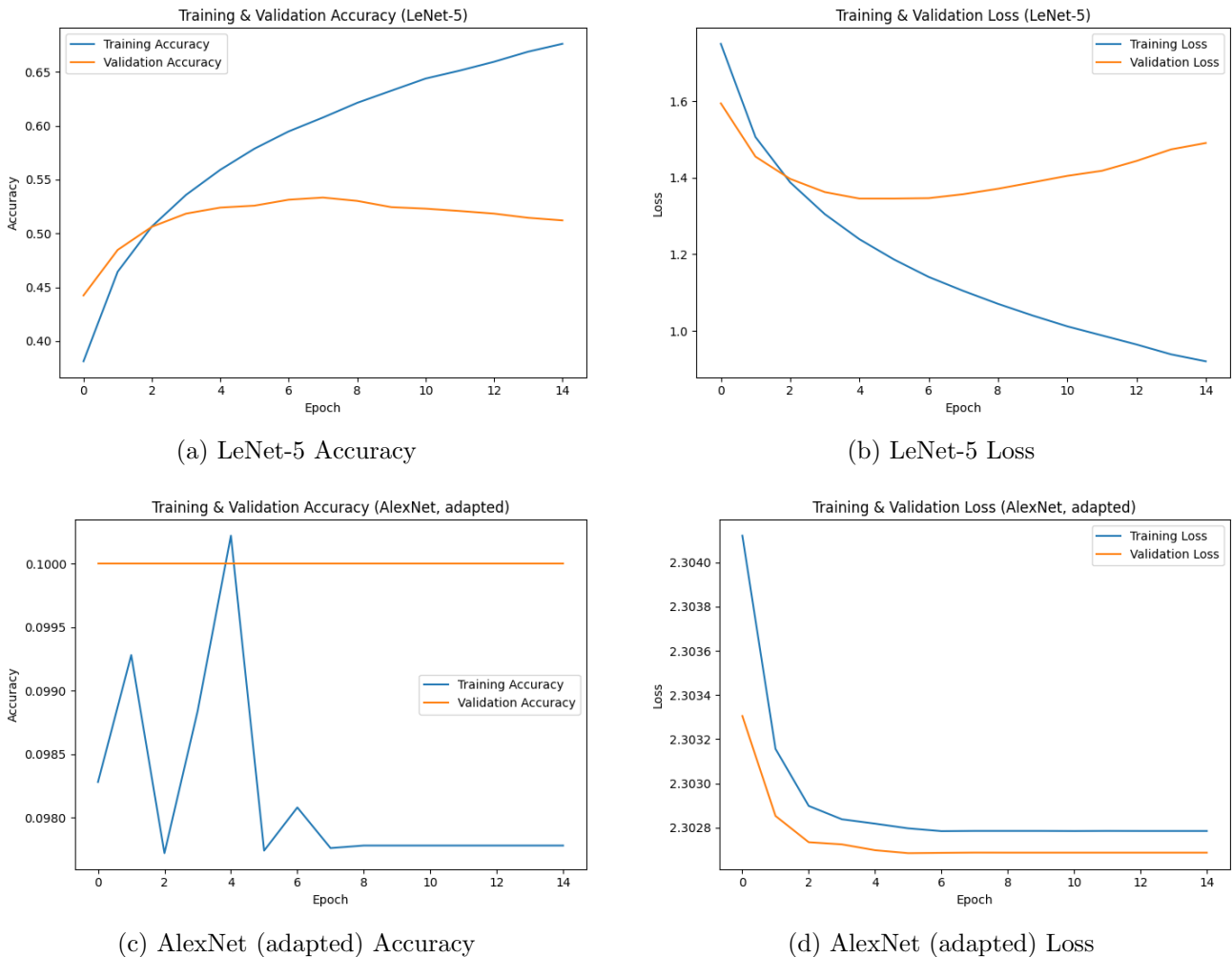


Figure 12: LeNet-5 and AlexNet Training Curves (trained from scratch)

Inference: LeNet-5, trained from scratch, reached 51.21% test accuracy in just 97.2 seconds – a decent result for such a tiny 7-layer, 83K-parameter network with no pretrained weights to rely on. Its validation accuracy peaks around epoch 8–9 (about 53%) and then drifts down a little while training accuracy keeps rising, so it starts overfitting slightly near the end. AlexNet (adapted), on the other hand, just didn’t train at all – its loss gets stuck at exactly 2.3028 (which is $\ln(10)$, the loss you’d get from pure random guessing over 10 classes) from epoch 1 all the way to epoch 15, and accuracy

never moves off 10%. This is a dead network – most likely because the architecture has no batch normalization anywhere and fairly large early filters, which pushed the ReLU units into a state where they stopped learning almost immediately. So this is a real result: the adapted AlexNet as built here did not converge on CIFAR-10 from scratch, and that’s reported as-is rather than fixed or swapped out.

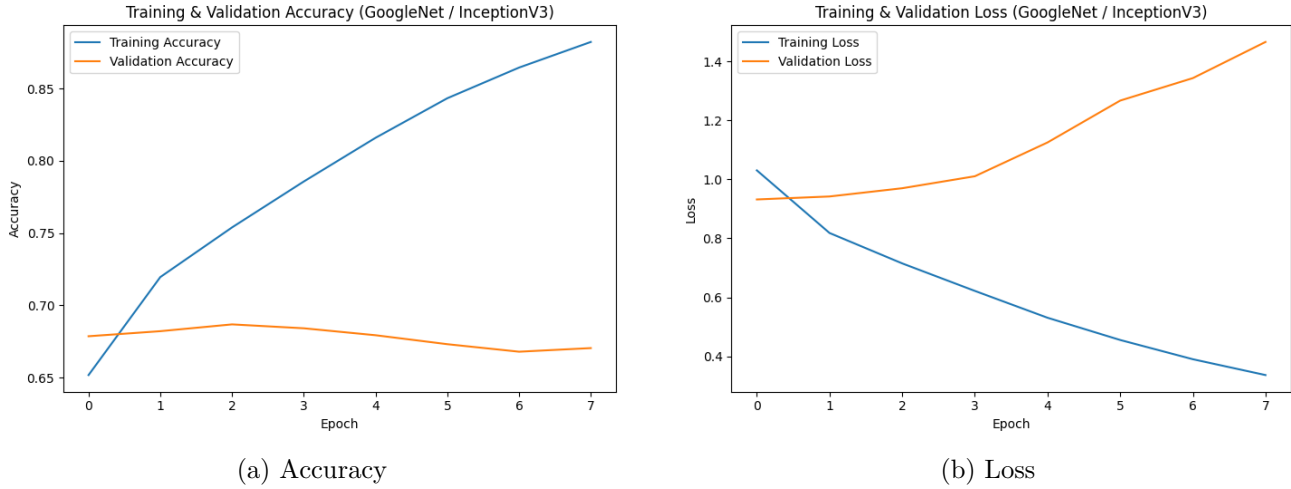


Figure 13: GoogleNet (InceptionV3) Training and Validation Accuracy/Loss

Inference: GoogleNet (InceptionV3, pretrained, frozen base) reached 67.04% test accuracy in 259.8 seconds over 8 epochs. Training accuracy keeps rising all the way to 88.22%, but validation accuracy actually peaks early, around epoch 3 (68.68%), and then slowly drops as validation loss climbs – another clear case of overfitting. This model only went through the frozen-base stage and wasn’t fine-tuned like VGG16 was, which is the main reason it trails behind VGG16 and ResNet50 – it never got the chance to adjust its higher-level features specifically for CIFAR-10.

19. Results

19.1 Performance Metrics (Primary Model – VGG16, fine-tuned)

Metric	Value
Training Accuracy	99.55%
Testing Accuracy	85.84%
Precision	85.96%
Recall	85.84%
F1-score	85.84%
Training Time	1825.3 s
Total Parameters	14,848,586

19.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	83,126	51.21	97.2
AlexNet (adapted)	8,576,906	10.00	396.8
VGG16	14,848,586	85.84	1825.3
GoogleNet (InceptionV3)	22,329,898	67.04	259.8
ResNet50	24,114,826	85.31	414.1

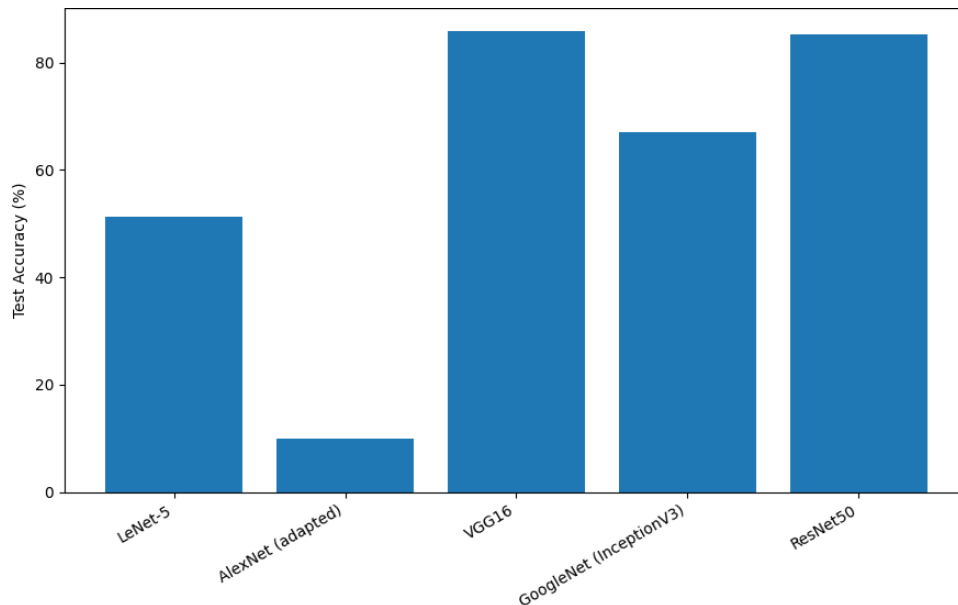


Figure 14: Accuracy Comparison – LeNet-5, AlexNet, VGG16, GoogleNet, ResNet50

Inference: Ranked by accuracy: VGG16 (85.84%) and ResNet50 (85.31%) come out on top and are basically tied, followed by GoogleNet/InceptionV3 (67.04%), then LeNet-5 (51.21%), and finally AlexNet-adapted (10.00%, which never actually trained). VGG16 and ResNet50 being so close makes sense since both are strong ImageNet-pretrained backbones, but ResNet50 got there over $4\times$ faster, showing what the residual connections are actually good for. GoogleNet falls behind mainly because it wasn't fine-tuned like VGG16 was. LeNet-5 is interesting – with only 83K parameters (way fewer than any of the other models) it still manages 51%, which shows that even a very small network can learn something on CIFAR-10, just nowhere near as much as a properly fine-tuned pretrained model. AlexNet-adapted's 10% isn't a real accuracy score at all – it's just the dead network discussed above stuck at random guessing.

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

It was the first deep CNN to win ImageNet by a big margin (2012), and it proved that deep networks trained on GPUs with ReLU activations and Dropout could beat traditional hand-engineered computer vision methods by a large margin. That result is basically what kicked off the whole deep learning wave in computer vision.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking a few 3×3 filters gives roughly the same receptive field as one bigger filter (two 3×3 layers cover about the same area as one 5×5), but with fewer parameters and more non-linear activations packed in between, which makes the network more expressive without blowing up its size.

3. **Explain the advantages of the Inception module.**

It runs several filter sizes (1×1 , 3×3 , 5×5) and pooling in parallel and then concatenates all the outputs, so it picks up features at multiple scales in a single layer. The 1×1 convolutions before the bigger filters cut down the number of channels first, which keeps the parameter count and computation manageable without hurting accuracy.

4. **What is the purpose of residual learning?**

Instead of learning the full mapping $H(x)$ directly, residual (skip) connections let the network learn just the residual $F(x) = H(x) - x$. This gives the gradient a direct path back through the network during backpropagation, so it doesn't vanish in very deep architectures, which is what makes training networks with 50+ layers actually feasible.

5. **Differentiate LeNet and ResNet.**

LeNet-5 (1998) is a small, shallow network with about 60K parameters and just 2 convolution layers, built for small grayscale digit images. ResNet (2015) is way deeper (50+ layers), uses skip connections to avoid vanishing gradients, and is trained on large, colour, ImageNet-scale datasets with millions of parameters – basically a completely different scale of model built for a completely different problem.

6. **What is Transfer Learning?**

It means taking a model that's already been trained on a large dataset like ImageNet and using it as a starting point for a new (usually smaller) dataset, instead of training from scratch. You get to reuse the general low- and mid-level features it already learned rather than relearning them from zero.

7. **Why is fine tuning required?**

When the base is frozen, only the new classifier head gets trained, so the features underneath stay generic to whatever dataset the model was originally trained on. Fine-tuning unfreezes some of the later convolutional layers so they can adjust to the new dataset specifically, which usually pushes accuracy up further – exactly what happened here, where fine-tuning took VGG16 from 81.84% to 85.84%.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated (atrous) convolution spaces out the kernel elements to widen the receptive field without adding more parameters or losing resolution – useful when you need more context, like in segmentation. Transpose convolution does the opposite kind of job: it's a learnable way to *upsample*, increasing the spatial size of a feature map, which is used in things like autoencoders and GANs to reconstruct or generate images.

9. **Why do pretrained models converge faster?**

Their weights already encode general, reusable visual features like edges, textures and shapes, learned from a huge dataset. So instead of starting from random weights, the model already starts fairly close to a good solution and just needs to adjust rather than learn everything from zero – which is exactly why LeNet-5 and AlexNet, both trained from scratch here, needed 15 epochs each and still ended up behind the pretrained models that trained for far fewer epochs.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet-5 is tiny (around 83K parameters, 2 conv layers), so it trains and runs almost instantly even on modest hardware – 97.2 seconds in this experiment. ResNet50 has about 24.1M parameters spread across 50 layers, which needs a lot more compute, memory, and ideally a GPU (414.1 seconds here). But that extra depth is only usable because of the residual connections – a plain 50-layer network without skip connections would likely struggle to train at all.

21. Additional Exercises

1. Implement transfer learning using VGG16. – Done in Tasks 2–5 (Section 15).
2. Repeat the experiment using ResNet50. – Done in Section 17.
3. Compare Adam and SGD optimizers. – Done in the Hyperparameter Study (Section 16); Adam clearly beat SGD (0.7175 vs 0.6335 validation accuracy) within the same 3-epoch budget.
4. Compare training with frozen layers and fine tuning. – Done in Tasks 3–4 and the Hyperparameter Study; fine-tuning raised accuracy from 81.84% to 85.84%, while partially unfreezing without a

proper fine-tuning setup collapsed accuracy to 10.85%.

5. Evaluate the model using Precision, Recall and F1-score. – Done in Task 5 (Section 15) and Section 19.1.
6. Compare the performance of LeNet, AlexNet and ResNet. – Done in Section 18 and the architecture comparison table in Section 19.2.

22. Source Code

The complete, executed Google Colab notebook, along with all 600 DPI figures and CSV result files generated during the run, is available on GitHub:

https:
[//github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_4_transfer-learning](https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_4_transfer-learning)

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>